RAG 系统设计实践指南

第一章 · 检索增强生成的基本原理

检索增强生成（Retrieval-Augmented Generation，简称 RAG）是一种把外部知识库与大语言模型结合起来的架构范式。它的核心思想是：在回答用户问题之前，先从知识库中检索相关文档片段，把这些片段作为上下文一起交给模型，从而让模型基于可溯源的事实作答，而不是仅凭训练时记住的参数化知识。

一个典型的 RAG 流水线包含五个阶段：加载（Load）、切块（Chunk）、向量化（Embed）、检索（Retrieve）与生成（Generate）。其中切块策略对最终效果影响极大：块太大则噪声多、召回精度低；块太小则上下文断裂、语义不完整。生产环境中常见的做法是根据文档结构做递归切分，并保留一定重叠以避免边界信息丢失。

评估 RAG 效果同样需要拆解。检索阶段关注召回率与准确率，生成阶段关注忠实度（faithfulness）与答案相关性。多数线上问题并非出在生成环节，而是检索阶段没有把正确的片段召回。因此在优化模型之前，先建立一套带标注的评测集，往往能取得更高的投入产出比。

文档切块方法对比

第二章 · 常用分块器一览

下表列出了几种常见分块器的核心参数与适用场景。实际选型时，应当先用真实业务文档做小规模实验，而不是照搬默认值。

分块器	切分依据	适用场景
CharacterTextSplitter	固定字符数	结构单一的纯文本
RecursiveCharacterTextSplitter	递归尝试多个分隔符	通用默认选择
TokenTextSplitter	Token 数量	需要精确控制模型上下文
MarkdownHeaderTextSplitter	Markdown 标题层级	结构化技术文档

向量数据库选型要点

第三章 · 存储层的工程考量

向量数据库的选型通常围绕四个维度展开：索引结构、过滤能力、写入性能与运维成本。

- 索引结构：HNSW 在召回率和查询延迟之间取得较好平衡，是目前的主流选择；IVF 适合超大规模数据，但需要重建索引。
- 过滤能力：支持在相似度检索的同时按 metadata 过滤（如按时间、租户隔离），是很多企业场景的硬性需求。
- 写入性能：需要频繁更新文档时，应关注索引的增量写入能力，避免每次全量重建。
- 运维成本：单机嵌入式数据库（如 Chroma、FAISS）部署简单；分布式方案（Milvus、Qdrant）提供高可用与水平扩展，但运维复杂度更高。
- 混合检索：在向量检索之外叠加关键词检索（BM25），往往能显著改善对专有名词、型号、编号类问题的召回效果。